Contents

1. Document History 3
2. API Overview 4
3. API Login 4
3.1. Access Token Login 5
3.2. Normal Login 6
4. Provider APIs 7
4.1. Open Game URL 8
4.2. List Game 9
4.3. Game Round Status 12
4.4. Game History URL 15
4.5. Balance 17
4.6. Withdraw 19
4.7. Verify Withdraw transaction 20
4.8. Get Withdraw/Deposit Transaction by date 23
4.9. Sign Out 25
4.10. Get Domain Whitelist 26
4.11. Register Domain Whitelist 27
4.12. Unregister Domain Whitelist 29
4.13. Tournament APIs 30
4.13.1 Open Tournament Lobby URL 30
4.13.2 Get List Tournaments 31
4.13.3 Get Tournament Info 35
4.13.4 Get Tournament Rank 38
4.13.5 Get Tournament Tickets 40
5. Operator Integration APIs 43
5.1. Authenticate Token 45
5.2. Authenticate Username/Password (Optional) 47
5.3. Balance 49
5.4. Gameplay APIs – Slot & Live Casino 50
5.4.1. Bet 51

5.4.2. Settle Bet 53
5.4.3. Cancel Bet 56
5.4.4. Bonus Win 59
5.4.5. Jackpot Win 61
5.5. Gameplay APIs – Fishing 63
5.5.1 Transaction 63
5.5.2 Withdraw 66
5.5.3 Deposit 68
5.6. Tournament APIs 70
5.6.1 Join Tournament 70
5.6.2 Cancel Join Tournament 72
5.6.3 Win Tournament 75
6 Appendix 77
6.1 Timestamp 77
6.2 Hash 77
6.3 Status Code 83
6.4 Datetime Format (Global Convention) 84
6.5 Language Format (Global Convention) 84
6.6 Delivery & Retry Mechanism 84
6.7 Game Type 85
6.8 Type & Description Mapping 86
7 FAQ 87

1. Document History

Date	Version	Description	By
2025-Sep-15	3.0.1	Add "Type" = "Promotion" to 6.8 Type & Description Mapping	Customer Support
2025-Aug-01	3.0.0	No changes to API logic — documentation enhancement only	Customer Support
2024-Jun-03	1.29	Add "template" parameter when accessing the game lobby	Customer Support
2024-Mar-04	1.27	Add **4.10 Get Domain Whitelist** API Add **4.11 Register Domain Whitelist** API Add **4.12 UnRegister Domain Whitelist** API	Customer Support
2023-Mar-20	1.26	Access to the game lobby	Customer Support
2022-Oct-19	1.25	Describe maximum length of token	Customer Support
2022-Aug-25	1.24	Add **5.5. Get Tournament Tickets** API	Customer Support
2021-Jul-07	1.23	Add **5. Tournament APIs** Add **7. Operator Integration Tournament APIs**	Customer Support
2021-May-05	1.21	Add **4.9. Sign Out** API	Customer Support
2021-Feb-18	1.20	**ID, RoundId**: Unique by member and only effect with new clients	Customer Support
2020-Nov-6	1.19	Extend the username length to **32** characters	Customer Support
2020-June-30	1.17	Get **Withdraw/Deposit** transactions	Customer Support
2020-April-20	1.15	Remove **extendedinfo**	Customer Support
2020-April-08	1.14	Add **extendedinfo** parameter for bet API	Customer Support
2020-Mar-27	1.13	The username: 4 to 20 alphanumeric characters and insensitive	Customer Support
2020-Feb-12	1.12	Add **extendedinfo** parameter for settle-bet API	Customer Support
2019-July-24	1.10	Add **balance**, **withdraw** API	Customer Support
2019-March-22	1.0	Create	Customer Support

2. API Overview

Common terms are used in the document:

- **Provider**: refers to API/System.
- **Operator**: refers to who needs to integrate with our system. Operator and Operator's system are synonymous terms in this document.

This document describes a set of APIs that the **Operator** must implement in order to fully integrate with the Seamless Wallet API, including request validation, balance checks, and transaction updates.

The **Provider** API and **Operator** Integration API use **a cryptographic hash (MD5)** to verify the integrity and authenticity of incoming requests. The detailed encryption method is described in **section 6.2 Hash**.

3. API Login

The provider supports 2 types of login method:

- Access Token Login (suitable for Website)
- Normal Login (suitable for Mobile App)

3.1. Access Token Login

Access Token login can be used when Operator hosts the provider's games in their lobby. For more details on how the token authentication works please refer to section **5.1 Authenticate Token**.

The login flow is as follows:

1. After Player logins on Operator system, Operator system passes the **Token** to open Game link:

```
GET {Forward-URL}/playGame?token={token}&appId={appID}&gameCode={gameCode}&
language={language}&mobile={isMobile}&redirectUrl={redirectUrl}&template={t
emplate}
```

   For more information on each parameter in this URL and how it is used to open the game, please refer to **section 4.1 Open Game URL**.

2. Game client calls the provider API which in return calls Operator Integration API to authenticate Token.
3. Operator Integration API responses to validate the token together with Username, Balance
4. Provider game site shows the game.

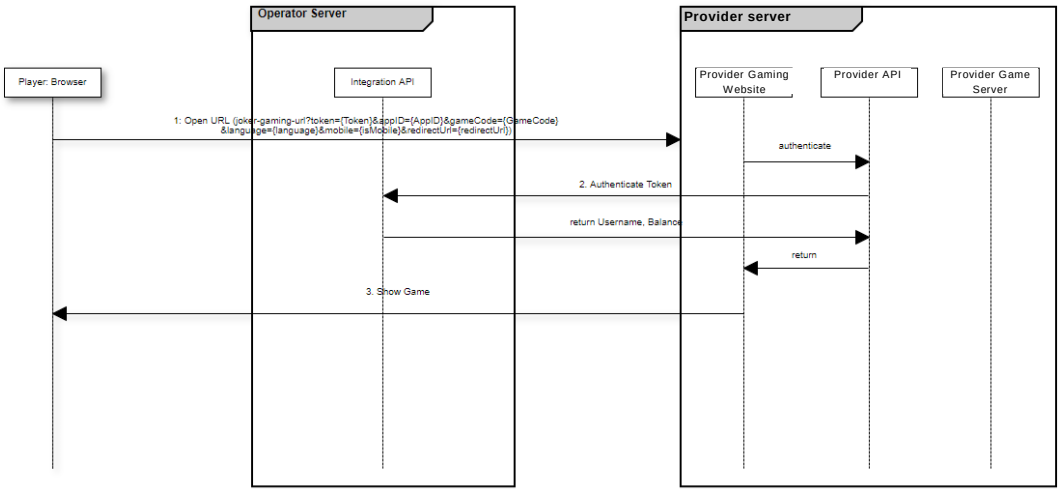

Diagram 1: Access Token Login

3.2. Normal Login

We provide a normal login option that allows the **Operator's** players to log in directly through the gaming platform, either via **Mobile App** or **Website**. For more details on how the authentication works and the description of each parameter, please refer to section **5.2 Authenticate Username/Password**.

To help the **Provider** determine which **AppId** the user belongs to, the **Username** must follow this format: **AppId.Username**

The login flow is as follows:

1. The **Player** accesses the gaming platform via mobile app or website, and logs in using their **username/password**.
2. The **Game client** calls the Provider API, which in turn calls the **Operator Integration API** to authenticate the username/password.
3. The **Operator Integration API** validates the credentials and returns the **Token**, **Balance**, and other relevant player information.
4. The **gaming platform** (mobile or web) displays the game interface.

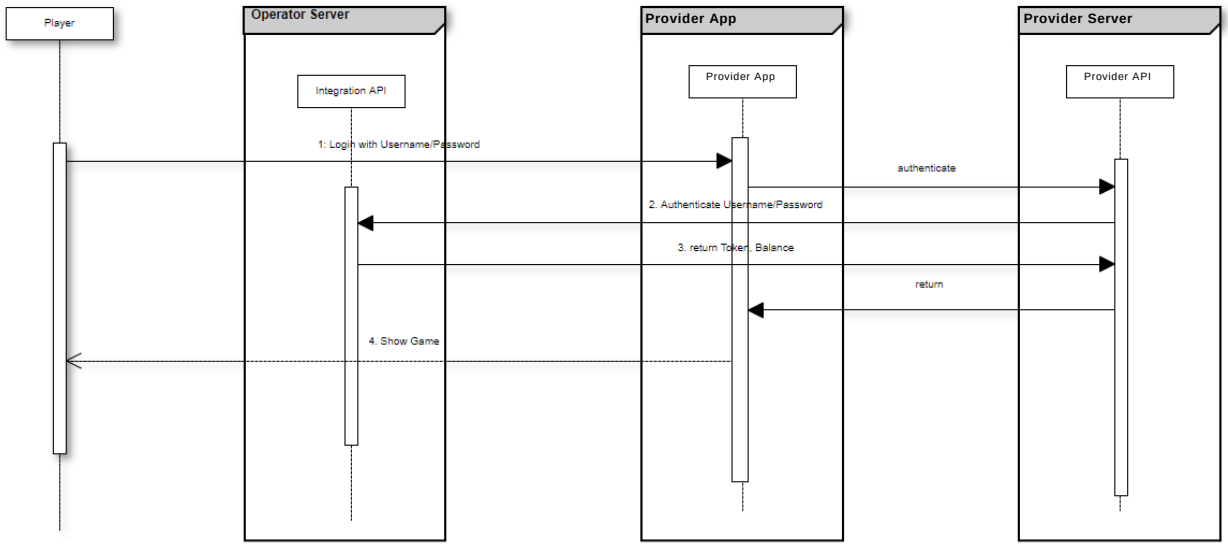

Diagram 2: Normal Login

4. Provider APIs

This section describes the set of APIs provided by the **Provider**, which are intended for integration by the **Operator**. Each API serves a specific purpose in the overall workflow of **launching**, **managing**, or **retrieving information** related to games.

Depending on the use case, the **Operator** may implement different APIs in this section. Detailed descriptions, required parameters, and expected responses are provided individually for each API.

General Behavior:

- **When the Operator sends a request to any of the Provider's APIs**:
  1. If the request is **accepted and processed successfully**, the Provider will always respond with **HTTP Status Code 200**.
  2. Any status **other than 200** should be considered as **request not accepted** or **connection** failure. The Provider includes a **Message** field in the response body to help with **debugging or error analysis**.
     Example:

```
Response body: HTTP/1.1 404 Not Found
{
    "Message": "App was not found"
}
```

- **Note on Timestamp**:
  1. All requests sent to **Provider** APIs must include a **Timestamp** parameter to indicate the moment the request was generated on the **Operator side**. This value must be in **UNIX timestamp format (milliseconds)** *please refer to section 6.1 Timestamp*.
  2. The **Provider** system will **reject requests if the timestamp is too old** (**more than 1 hour difference** between the request time and the **Provider** server's current time), in order **to prevent replay attacks or delayed transmissions**.

4.1. Open Game URL

This API is called by the **Operator** system to open the **Provider's game** after the player has successfully logged in on the **Operator's** website.

The system constructs a **URL** with all the **required information** (e.g., **token**, **gameCode**) and redirects the player to that URL to **launch** the game session.

```
GET {Forward-URL}/playGame?token={token}&appId={appID}&gameCode={gameCode}&
language={language}&mobile={isMobile}&redirectUrl={redirectUrl}&template={te
mplate}
```

Property	Type	Compulsory	Description
appId	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
token	String	Y	**Generated** by **Operator**. Used to uniquely identify the logged-in player. *(Token has max length 64 characters)* Note: Please encode the value with UrlEncode to avoid unexpected error
gameCode	String	Y	Game code as listed by **Provider**. If this value is **empty**, the player will be **redirected** to the default **game lobby** instead. *Please refer to section 4.2 List Game API*
language	String	N	Player's preferred language, using ISO 639-1 (2-letter) code. If not provided, the **default** value is **en**. *Please refer to section 6.5 – Language Format (Global Convention) for supported values*
mobile	Boolean	N	Indicates whether the client is on mobile. Accepted values: **true** or **false** If not provided, the **default** value is **false**.
redirectUrl	String	N	The URL to redirect the player after logout. Optional for web-based flow. If left empty, the game interface will not

			display a sign-out button Note: Please encode the value with UrlEncode to avoid unexpected error
template	String	N	The game lobby opens with a template - Howdy-Infinity - Howdy-Utopia - Howdy-Ruby - Howdy-Space - Howdy-Amethyst - Howdy-Navy

4.2. List Game

Returns the list of **games** and their associated resources. Games with a lower **Order** value will be displayed on top — these are typically the **newer** and more **popular** games.

The Order value may change dynamically as new games are added or the popularity of existing games changes.

API Specification:

```
API URL: {api-url}/list-games
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "Hash": "792d2414e3b0c77e9ddea9e8aa924364",
    "Timestamp": 1528210236000
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "ListGames": [
        {
            "GameType": "Slot",
            "GameCode": "dxxsh3dfmjpio",
            "GameName": "Tai Shang Lao Jun",
            "SupportedPlatForms": "Desktop,Mobile",
            "Specials": "hot,new",
            "Order": 1,
            "FreeSpin": true,
            "DefaultWidth": 960,
            "DefaultHeight": 630,
            "OriginalImage1":
```

```
"//sampledomain/gameimages/landscape/fwria11mjbrwh.png",
            "OriginalImage2":
"//sampledomain/gameimages/portrait/fwria11mjbrwh.png",
            "Image1":
"//sampledomain/gameimages/landscapeBonus/taishanglaojungw_99.png",
            "Image2":
"//sampledomain/gameimages/portraitBonus/taishanglaojungw_99.png"
        }
    ]
}
```


Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
ListGames	Array<Object>	List of available games returned by the API.

Game Object Fields:

Property	Type	Description
GameCode	String	**Unique identifier** of the game
GameName	String	Display name of the game shown to users
DefaultWidth	Integer	Default width of the game window (in pixels). Example: 1280.
DefaultHeight	Integer	Default height of the game window (in pixels). Example: 720
GameType	String	Type of game. *See section 6.7 GameType for definition*.
SupportedPlatForms	String	Platforms supported by the game (comma-separated), e.g., Desktop, Mobile.
Specials	String	Comma-separated tags used for highlighting games in the client interface. **Supported values:** • recommend: featured/recommended game • new: newly launched game. • hot: hot game.
Order	Integer	Display priority. Games with a lower Order value appear earlier in the list.
FreeSpin	Boolean	Indicates whether the game supports the Free Spin feature • True: The game supports Free Spin. • False: The game does not support Free Spin.
OriginalImage1	String	URL of the original landscape image for the game.
OriginalImage2	String	URL of the original portrait image for the game.
Image1	String	URL of the original landscape image for the game. If the game supports bonuses, this image may include bonus icon overlays.
Image2	String	URL of the original portrait image for the game. If the game supports bonuses, this image may include bonus icon overlays.

4.3. Game Round Status

This API allows the **Operator** to query the current status of a specific game round.

It is typically used in scenarios where the round has already been accepted by the Operator but has not yet been settled within **24 hours** of acceptance. By calling this API, the **Operator** can confirm whether the round has been processed or requires **additional handling**.

This API only is used for game rounds that were triggered from the Bet API. For more details, please refer to section 5.4.1 Bet API.

Additional Notes:

1. Rounds that have been **successfully settled** on the **Provider system** cannot be **queried after 30 days**.
2. Rounds that have been **successfully cancelled** on the **Provider system** cannot be **queried after 3 days**.
3. Rounds that have been **accepted** and are still waiting for the **player to log in** to complete the game round will return with **Status = "Success"**.
   ⇨ *There is no time limit for querying rounds in this status.*
4. If the **Operator calls this API** after the **time limits** mentioned above (**30 days** for settled rounds, **3 days** for cancelled rounds), the API will not return any game round status. **The response will be**:

```
Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "Bets": []
}
```

5. If the above response is returned **before** the time limit has been reached, please contact our **Support Team** to **investigate the issue**. Thank you.

API Specification:

```
API URL: {api-url}/game-round-status
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "RoundID": "uhax9gb5upqgq",
    "Username": "DEMO",
    "Hash": "792d2414e3b0c77e9ddea9e8aa924364",
    "Timestamp": 1528210236000
```

```
}

Response body: HTTP/1.1 200 OK
1. Empty Result (No data available)
{
    "Error": "0",
    "Description": "OK",
    "Bets": []
}

2. Round Cancelled
{
    "Error": "0",
    "Description": "OK",
    "Bets": [
        {
            "ID": "B_Tde8c51866e1b_G2ad04662c628",
            "Status": "Cancelled"
        }
    ]
}

3. Round Accepted, Waiting for Player Login
{
    "Error": "0",
    "Description": "OK",
    "Bets": [
        {
            "ID": "B_T1a5c664df65b_G2ad050b778b8",
            "Status": "Success"
        }
    ]
}

4. Round Settled
{
    "Error": "0",
    "Description": "OK",
    "Bets": [
        {
            "ID": "xeuzyaa5f9qnk",
            "Status": "Settled",
            "Amount": 5.00,
            "Result": 0.50,
            "Time": "2025-08-04T15:53:42.905+08:00"
        }
    ]
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
RoundID	String	Y	**Unique identifier** of the game round, used to trace the specific transaction *Please refer to section 5.4.1 Bet API*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
Username	String	Y	Username of the player associated with the game round. **Do not use this property in Hash**

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Bets	Array<Object>	A list of all bets associated with the queried game round. Each item in this array includes the bet ID and its processing status.

Bet Object Fields:

Property	Type	Description
ID	String	**Unique identifier** of the bet – **BetID** *Please*

		refer to section 5.4.1 Bet API **Note**: If the round has been successfully **settled**, this field will return the **RoundID** instead of **BetID**.
Status	String	Current processing status of the game round. Possible values: • **Success** – Round accepted and awaiting player login to complete. • **Cancelled** – Round has been cancelled and will not be processed. • **Settled** – Round has been completed and settled successfully.
Amount	Decimal	Betting amount submitted in the **Bet API** *Please refer to section 5.4.1 Bet API*
Result	Decimal	Result amount returned by the **Settle Bet API**. *Please refer to section 5.4.2 Settle Bet API*
Time	Datetime	The timestamp indicating when the round was successfully settled on the Provider system. *Please refer to see section 6.4 – Datetime Format (Global Convention) for format details.*

4.4. Game History URL

This API allows the **Operator** to retrieve the detailed game history of a specific round.

Notes:

1. **Only games listed** in **Section 4.2 – List Game** are eligible for **game history retrieval**.
2. **Only settled rounds** — i.e., those successfully completed via the **Settle Bet API** or **Transaction API** — are available for **history viewing**.
3. **Ineligible rounds** include those with the following **statuses**:
    a. **Pending** – Round is awaiting player login or action to complete.
    b. **Cancelled** – Round was cancelled and will not proceed.

API Specification:

```
API URL: {api-url}/game-history-url
HTTP Method: POST
Content-Type: application/json


Request body:
{
    "AppID": "AppID",
    "Username": "DEMO",
```

```
    "GameCode": "xea1rt9gebhwy",
    "RoundID": "uhax9gb5upqgq",
    "Language": "en",
    "Hash": "792d2414e3b0c77e9ddea9e8aa924364",
    "Timestamp": 1528210236000
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "Url":
"//History?Signature=k2Nyoas94qHkAaoIKuYAdJEnr9c%3d&Key=xkqryq3b5aeuq&Type=G
ame&Timestamp=1552962025093&Lang=en"
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
RoundID	String	Y	**Unique identifier** of the game round, used to trace the specific transaction *Please refer to section 5.4.1 Bet API*
GameCode	String	Y	Game code used to identify which game the round belongs to. Only games listed in section 4.2 List Game are eligible.
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
Username	String	Y	Username of the player associated with the game round.
language	String	Y	Player's preferred language, using ISO 639-1 (2-letter) code. If not provided, the **default** value is **en**.

	Please refer to section 6.5 – Language Format (Global Convention) for supported values

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Url	String	The URL used to view the game history detail. *This URL is valid for 10 minutes after it is generated and will expire automatically thereafter.*

4.5. Balance

This API allows the **Operator** to check the current credit balance of a player on the **Provider** system. It is used by **Operators** who support deposit and withdrawal through **5.5.2 – Withdraw API** and **5.5.3 – Deposit API.**

API Specification:

```
API URL: {api-url}/balance
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "Username": "DEMO",
    "Hash": "792d2414e3b0c77e9ddea9e8aa924364",
    "Timestamp": 1528210236000
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "Amount": 100.01
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Username	String	Y	Player's username
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Value	Description
Error	String		API result status: • "0" = Success • "1" = Failure
Description	String		Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Amount	Decimal	Positive numeric	The current available balance on the **Provider** system. Value must be a positive decimal number.

4.6. Withdraw

The API allows the **Operator** to trigger the **5.5.3 Deposit API**, which is used to return money back from the **Provider's** system to the **Operator's** system.

When triggered, the **5.5.3 Deposit API** will automatically withdraw **the entire available balance** of the player from the **Provider's** system and transfer it to the **Operator's** system.

API Specification:

```
API URL: {api-url}/withdraw
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "Username": "DEMO",
    "Hash": "792d2414e3b0c77e9ddea9e8aa924364",
    "Timestamp": 1528210236000
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK"
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Username	String	Y	Player's username
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Value	Description
Error	String		API result status: • "0" = Success • "1" = Failure
Description	String		Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.

4.7. Verify Withdraw transaction

This API allows the **Operator** to verify the processing status of a specific **Withdraw** request that was previously triggered by the **Provider** via the **5.5.2 Withdraw API**.

Notes:

1. This API is particularly useful in cases where the **Operator** has already deducted the balance from the player, but the **Provider** did not record the Withdraw request in its system — for example, due to a **timeout** or **network issue**.
2. By calling this API, the **Operator** can confirm whether the withdraw request was actually received and processed by the **Provider** system.

API Specification:

```
API URL: {api-url}/statement-by-request
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "Username": "DEMO",
    "AppID": "AppID",
    "IDs": " 5ee99dcba3787, 5ee98746d9216",
    "Timestamp": 1593144363603,
    "Hash": "abb7cc87ea3914e46c3a9562dfe89832"
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "Data": [
        {
            "Id": "5ee99dcba3787",
            "Username": "AppID.DEMO",
```

```
            "Time": "2020-06-24T15:38:55.169",
            "Amount": 20.00,
            "Type": "TopUp"
        },
        {
            "Id": "5ee98746d9216",
            "Username": "AppID.DEMO",
            "Time": "2020-06-24T16:38:55.169",
            "Amount": 30.00,
            "Type": "TopUp"
        }
    ]
}
```


Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Username	String	Y	The username of the player whose withdrawal transactions are being verified.
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
IDs	String	Y	A list of request IDs that were originally generated from the *5.5.2 Withdraw API*. Multiple IDs should be comma- separated Example: **"5ee99dcba3787,5ee98746d9216"**

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success

		• "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Data	Array<Object>	Array of transaction records matching the requested IDs **Note**: If the **ID** does not exist or does not belong to the specified **Username** this array will be empty.

Array Object Fields:

Property	Type	Description
Id	String	The transaction ID corresponding to the request from *the Withdraw API section 5.5.2*
Username	String	The username of the player associated with this transaction. Format: **AppID.Username**
Time	DateTime	The time the transaction was successfully processed *See section 6.4 – Datetime Format (Global Convention) for format details.*
Amount	Decimal	The amount in this transaction
Type	String	The type of transaction: **TopUp** **TopUp** – Indicates a **deposit request into the Provider system**. This means the **Provider has triggered** *the Operator's Withdraw API section 5.5.2* in order to credit the player's balance on the **Provider** side.

4.8. Get Withdraw/Deposit Transaction by date

This API allows the **Operator** to retrieve the complete list of successful **Withdraw** *(see section 5.5.3 Deposit* and **Deposit** *(see section 5.5.2 Withdraw)* transactions recorded on the **Provider** system for a specified date.

API Specification:

```
API URL: {api-url}/statement-by-date
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "Username": "DEMO",
    "AppID": "AppID",
    "Date": "2020-06-24",
    "Timestamp": 1593498325050,
    "Hash": "edc6c90c32b1e148d9c2bbf4f4eeb311"
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "Data": [
        {
            "Id": "5ee99dcba3787",
            "Username": "AppID.DEMO",
            "Time": "2020-06-24T15:38:55.169",
            "Amount": 20.00,
            "Type": "TopUp"
        },
        {
            "Id": "5ee98746d9216",
            "Username": "AppID.DEMO",
            "Time": "2020-06-24T15:40:04.152",
            "Amount": -3594.90,
            "Type": "Withdraw"
        }
    ]
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.

Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Username	String	Y	The player's username.
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
Date	String	Y	Date to query transaction history. **Format: yyyy-MM-dd (**`2020-06-24`)

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Data	Array<Object>	List of transaction entries matching the request date.

Array Object Fields:

Property	Type	Description
Id	String	**Unique identifier** of the transaction.
Username	String	Username of the player in format **AppID.Username**.
Time	DateTime	The time the transaction was successfully processed *See section 6.4 – Datetime Format (Global Convention) for format details.*
Amount	Decimal	Transaction amount. **Positive** = amount **deposited into** the **Provider** system **Negative** = amount **withdrawn from** the **Provider** system

Type	String	Type of transaction: • **"TopUp"** – Indicates a **Deposit** request (triggered via *5.5.2 Withdraw*) • **"Withdraw"** – Indicates a **Withdraw** request (triggered via *5.5.3 Deposit*)

4.9. Sign Out

This API allows the **Operator** to log the player out of the **Provider** system. Once the API is called successfully, the user will no longer be able to launch or continue any games on the **Provider** system.

API Specification:

```
API URL: {api-url}/sign-out
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "Username": "Username",
    "Hash": "792d2414e3b0c77e9ddea9e8aa924364",
    "Timestamp": 1620184560896
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "Sign out successfully."
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Username	String	Y	Player's username.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds.

	Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.

4.10. Get Domain Whitelist

This API allows the **Operator** to retrieve the list of whitelisted domains associated with a specific **AppID**.

API Specification:

```
API URL: {api-url}/get-domain
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "Hash": "792d2414e3b0c77e9ddea9e8aa924364",
    "Timestamp": 1620184560896
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "Domain": ["demo12.com", "www.demo.net"]
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to

			identify which game environment the player belongs to.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Domain	Array<string>	A list of whitelisted domains

4.11. Register Domain Whitelist

This API allows the **Operator** to register (whitelist) domain names that are permitted to launch games from the **Provider** system.

Notes:

1. A maximum of **100 domains per AppID** is allowed.
2. Domains must be submitted as a **comma-separated list**.
3. Only **whitelisted domains** can be used to launch or access games on the **Provider** platform.

API Specification:

```
API URL: {api-url}/register-domain
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "Domain": "demo.com,www.demo.net",
    "Hash": "792d2414e3b0c77e9ddea9e8aa924364",
    "Timestamp": 1620184560896
```

```
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "Ok"
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Domain	String	Y	List of domains separated by commas
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.

4.12. Unregister Domain Whitelist

This API allows the **Operator** to remove specific domain(s) from the whitelist associated with an **AppID**.

Notes:

1. Domains must be provided as a **comma-separated list**.
2. Once removed, the specified domains will no longer be allowed to launch or access any games from the **Provider** system.
3. This helps enforce strict domain access control for **security and regulatory compliance**.

API Specification:

```
API URL: {api-url}/unregister-domain
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "Domain": "demo.com,www.demo.net",
    "Hash": "792d2414e3b0c77e9ddea9e8aa924364",
    "Timestamp": 1620184560896
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "Ok"
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Domain	String	Y	List of domains separated by commas
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Compulsory	Description
Error	String	Y	API result status: • "0" = Success • "1" = Failure
Description	String	Y	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.

4.13. Tournament APIs

The **Tournament APIs** provide **Operators** with a complete set of endpoints to integrate tournament-related features into their gaming platforms. These APIs allow the **Operator** to open the tournament lobby, retrieve tournament listings, view detailed tournament information, track player rankings.

4.13.1 Open Tournament Lobby URL

Launches the **tournament lobby interface** provided by the **Provider**.

```
GET {Forward-URL}/playGame?token={token}&appID={appID}&gameCode=Tournament&
language={language}&mobile={isMobile}&redirectUrl={redirectUrl}&template={te
mplate}
```

Property	Type	Compulsory	Description
appID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
token	String	Y	Generated by **Operator**. Used to uniquely identify the logged-in player. **Token has max length 64 characters** **Note: Please encode the value with UrlEncode to avoid unexpected error**
gameCode	String	Y	Fixed value: **Tournament** – used to launch the tournament lobby.
language	String	N	Player's preferred language, using ISO 639-1 (2-letter) code.

			See section 6.5 – Language Format (Global Convention) for supported values
mobile	Boolean	N	Indicates whether the client is on mobile. Accepted values: **true** or **false** If not provided, the **default** value is **false**.
redirectUrl	String	N	The URL to redirect the player after logout. Optional for web-based flow. *If left empty, the game interface will not display a sign-out button* **Note: Please encode the value with UrlEncode to avoid unexpected error**
template	String	N	The game lobby opens with a template - Howdy-Infinity - Howdy-Utopia - Howdy-Ruby - Howdy-Space - Howdy-Amethyst - Howdy-Navy

4.13.2 Get List Tournaments

This API returns a list of available tournaments on the **Provider** system based on the specified **status**. Only tournaments created within the **last 7 days** are included in the response.

API Specification:

```
API URL: {api-url}/tournaments
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "Status": "All",
    "Timestamp": 1593498325050,
    "Hash": "155786eaa5211c74b45ab371c97b86ca"
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "Data": [
        {
            "TournamentID": "xeahz7p7oag51",
            "TournamentName": "TaiShangLaoJun",
```

```
            "StartDate": "2021-07-07T20:00:00",
            "EndDate": "2021-07-08T22:00:00",
            "Description": "Tai Shang Lao Jun",
            "Status": "New",
            "CurrencyCode": "MYR",
            "Prize": [
                {
                    "PrizeOCode": "nknny55cpitgg",
                    "Amount": 10000,
                    "Qty": 1,
                    "Position": 1,
                    "Ticket": 1
                },
                {
                    "PrizeOCode": "nknny55cpitgg",
                    "Amount": 5000,
                    "Qty": 2,
                    "Position": 2,
                    "Ticket": 1
                }
            ],
            "Cost": 1125,
            "RequiredTicket": 0,
            "ImageLink":
"http://dl.changxingwnet.com/tournament/assets/icon/248x248/5/TaiShangLaoJun
GW.png",
            "ImageLinkSmall":
"http://dl.changxingwnet.com/tournament/assets/web-
icon/320x265/1/TaiShangLaoJunGW.png",
            "Type": "total_win",
            "GameCode": "dxxsh3dfmjpio",
            "PrizePool": 39000,
            "NoOfSpins": 250,
            "TypeName": "Total Win"
        }
    ]
}
```


Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Status	String	Y	Filter for tournament status. Accepted values: • **All** – include all tournaments • **New** – recently created

			• **Active** – currently ongoing • **Completed** – already ended
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Data	Array<Object>	A list of Tournaments

Tournament Object Fields:

Property	Type	Description
TournamentID	String	**Unique identifier** for the tournament.
TournamentName	String	Display name of the tournament.
StartDate	DateTime	Start date and time of the tournament in ISO 8601 format. *See section 6.4 – Datetime Format (Global Convention) for format details.*
EndDate	DateTime	End date and time of the tournament in ISO 8601 format. *See section 6.4 – Datetime*

		Format (Global Convention) for format details.
Description	String	Text description or summary of the tournament event.
Status	String	Current status of the tournament. Possible values: **New**, **Active**, **Completed**.
CurrencyCode	String	Currency used in the tournament (e.g., **MYR**, **THB**, **USD**).
Cost	Decimal	**Entry fee** for joining the tournament. A non-zero value means that the player must pay this amount from their balance to participate.
RequiredTicket	Integer	Determines how players join the tournament: • **0** – Players **cannot** use an **ExclusivePass** to join. • **1** – Players **must own** an **ExclusivePass**, a special ticket distributed via in-game events such as **Angbao**, random drops, etc.
ImageLink	String	Full-size image URL representing the tournament
ImageLinkSmall	String	Small-size image URL representing the tournament
Type	String	Defines the **tournament calculation method**. Supported values: • **total_win** – Every score you wins in a single spin will be accumulated to the total. • **power_play** – the tournament is exclusive to **Buy Bonus** or **Freespins Feature**. The higher total score, the higher ranking is
GameCode	String	Identifier of the game associated with the tournament. *Refer to section 4.2 – List Game*
PrizePool	Decimal	Total **prize amount** of the tournament.

NoOfSpins	Integer	The number of spins each player is allowed to perform in the tournament, based on the **minimum bet level**.
TypeName	String	The user-friendly display name of the tournament type. It is shown in the UI. Supported values: • **Total Win** • **Power Play**
Prize	Array<Object>	A list of Prizes

Property	Type	Description
PrizeOCode	String	**Unique identifier** for the prize tier. Used for **tracking** purposes
Amount	Decimal	The **reward amount** granted to winners at this prize level.
Qty	Integer	Number of **winners** who will receive this prize.
Position	Integer	Player's ranking required to win this prize. For example: **1** = 1st place.
Ticket	Integer	Number of **ExclusivePass** granted to the winner, if applicable. Set to **0** if not used.

4.13.3 Get Tournament Info

This API retrieves detailed information for a specific tournament.

API Specification:

```
API URL: {api-url}/tournament
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "TournamentID": "xeahz7p7oag51",
    "Timestamp": 1593498325050,
    "Hash": "1936d0c41de7d522501b6b028f4fa178"
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
```

```
    "Description": "OK",
    "Data": {
        "TournamentID": "xeahz7p7oag51",
        "TournamentName": "TaiShangLaoJunGW",
        "StartDate": "2021-07-07T20:00:00",
        "EndDate": "2021-07-08T22:00:00",
        "Description": "Tai Shang Lao Jun",
        "Status": "New",
        "CurrencyCode": "MYR",
        "Prize": [
            {
                "PrizeOCode": "nknny55cpitgg",
                "Amount": 10000.0,
                "Qty": 1,
                "Position": 1,
                "Ticket": 1,
                "Points": 1000000
            },
            {
                "PrizeOCode": "nknny55cpitgg",
                "Amount": 5000.0,
                "Qty": 2,
                "Position": 2,
                "Ticket": 1,
                "Points": 500000
            },
            {
                "PrizeOCode": "nknny55cpitgg",
                "Amount": 3000.0,
                "Qty": 3,
                "Position": 3,
                "Ticket": 1,
                "Points": 300000
            },
            {
                "PrizeOCode": "nknny55cpitgg",
                "Amount": 1000.0,
                "Qty": 10,
                "Position": 4,
                "Ticket": 1,
                "Points": 100000
            }
        ],
        "Cost": 1125.00,
        "RequiredTicket": 0,
        "ImageLink":
"http://dl.changxingwnet.com/tournament/assets/icon/248x248/5/TaiShangLaoJun
GW.png",
        "ImageLinkSmall":
"http://dl.changxingwnet.com/tournament/assets/web-
icon/320x265/1/TaiShangLaoJunGW.png",
        "Type": "total_win",
```

```
        "GameCode": "dxxsh3dfmjpio",
        "PrizePool": 39000.0,
        "NoOfSpins": 250,
        "TypeName": "Total Win"
    }
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
TournamentID	String	Y	**Unique identifier** of the tournament to be retrieved.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Data	Object	Tournament detail object. The structure of this object is identical to each item in the Data array returned by *API 4.13.2 Get List Tournaments*. Refer to that section for full field definitions.

4.13.4 Get Tournament Rank

This API retrieves the current ranking list of players participating in a specific tournament.

API Specification:

```
API URL: {api-url}/tournament-rank
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "TournamentID": "xeahz7p7oag51",
    "PageIndex": 0,
    "Size": 20,
    "Timestamp": 1593498325050,
    "Hash": "4492ab519ea0038e2651ea40abc236a5"
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "Data": [
        {
            "Username": "0000000A012",
            "DisplayName": "Tester",
            "CurrencyCode": "MYR",
            "Chance": 40770,
            "Order": 1,
            "PrizeName": "First Prize",
            "WinningAmount": 10000,
            "IsWinner": true,
            "JoinTime": "2021-06-18T16:23:23.242",
            "Tickets": 0
        }
    ]
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the

			player belongs to.
TournamentID	String	Y	**Unique identifier** of the tournament for which the ranking is requested. Must match a value from *4.13.2 Get List Tournaments API*.
PageIndex	Integer	Y	Zero-based index indicating which page of ranking results to retrieve. For example, **0** = first page.
Size	Integer	Y	Number of ranking records to return per page. **Recommended default: 100**.
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Request Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Data	Array<Object>	A list of Rankings

Ranking Object Fields:

Property	Type	Description
Username	String	**Name of the prize winner.** If the player does not belong to the current **AppID**, this field will return the same value as **DisplayName**
DisplayName	String	Player's nickname or display name, shown

		in the ranking UI.
CurrencyCode	String	Currency in which the player is participating (e.g., MYR, THB).
Chance	Integer	Player's **total accumulated tournament points**.
Order	Integer	Player's current **rank** in the tournament (e.g., **1 = 1st place**).
PrizeName	String	Name of the prize awarded to the player (if any), based on current rank.
WinningAmount	Decimal	**Monetary prize amount** currently associated with the player's rank.
IsWinner	Boolean	Indicates whether the player is currently eligible for a prize. **true** = receiving prize **false** = not in prize zone.
JoinTime	Datetime	Date and time when the player joined the tournament. *See section 6.4 – Datetime Format (Global Convention) for format details.*
Tickets	Integer	Number of **ExclusivePasses** awarded to the player as part of the tournament prize.

4.13.5 Get Tournament Tickets

This API retrieves a list of **Exclusive Passes** that have been issued to a specific user.

API Specification:

```
API URL: {api-url}/tournament-ticket
HTTP Method: POST
Content-Type: application/json

Request body:
{
    "AppID": "AppID",
    "Username": "Username",
    "Timestamp": 1593498325050,
    "Hash": "4492ab519ea0038e2651ea40abc236a5"
}

Response body: HTTP/1.1 200 OK
{
    "Error": "0",
    "Description": "OK",
    "Data": [
```

```
        {
            "Status": "Used",
            "ValidFrom": "2021-08-02T12:36:34.789424",
            "ExpireAt": "2021-09-01T12:36:34.789424"
        }
    ]
}
```

Request Body:

Property	Type	Compulsory	Description
AppID	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
Username	String	Y	The player's username
Hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
Timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Description
Error	String	API result status: • "0" = Success • "1" = Failure
Description	String	Message string describing the result or error. Mainly used for debugging or logging purposes when API call fails.
Data	Array<Object>	A list of Tickets

Ticket Object Fields:

Property	Type	Description
Status	String	Current status of the Exclusive Pass.

		Supported values: • **Valid** – The pass is active and can be used to join a tournament. • **Used** – The pass has already been consumed and cannot be reused.
ValidFrom	Datetime	The datetime when the **Exclusive Pass** becomes valid. *See section 6.4 – Datetime Format (Global Convention) for format details.*
ExpireAt	Datetime	The datetime when the **Exclusive Pass** expires and can no longer be used. *See section 6.4 – Datetime Format (Global Convention) for format details.*

5. Operator Integration APIs

This section defines a set of APIs that must be implemented by the **Operator** on their server as part of the integration process with the **Provider's** gaming system.

APIs are required to be ***idempotent***, meaning that sending multiple identical requests — with exactly the same parameters, payload — will have **the same effect as sending the request once**.

For example, if a request to debit a wallet is sent twice with completely identical data, the wallet balance should **only be reduced once**.

All API calls must receive a response within **2.5 seconds**, otherwise they will be considered as failed due to timeout.

General Conventions: All **Operator** APIs must conform to the following standards

- **Security Requirements:**
  - All requests must include:
    - **timestamp**: Current UNIX time in milliseconds see *Section 6.1 – Timestamp*.
    - **hash**: MD5 signature generated from sorted parameters using a shared secret key see *Section 6.2 – Hash*
  - These are used for validation and replay protection.

- **Standard Request Format:**
  - All APIs must use the **POST** method exclusively.
  - Other methods (**GET**, **PUT**, **DELETE**, etc.) are **not allowed**.
  - **Content-Type:** application/x-www-form-urlencoded
    - All parameters are **URL-encoded** by the Provider before sending.
    - The **Operator is responsible** for decoding the parameter values.
- **Standard Response Format:**
  - HTTP status code should **always be 200 OK**.
  - **Content-Type**: application/json
  - Every API must return a valid JSON response with this structure:

```
{
  "Message": "Success",
  "Status": 0
}
```

  - **Status = 0**: Success
  - Any non-zero Status: Indicates failure; Message must describe the error.
  - See *Section 6.3 – Status Code* for more details.
- **Balance and Decimal Precision Rules**
  - The system supports up to **two decimal places** for balance amounts.

- The supported decimal precision depends on the currency type, based on its **conversion ratio** between **Real Currency**, **Back Office (BO) balance**, and **Game Points**:
  - **THB**: 1 Real THB = 1 BO = 100 Game Points
  - **MMK**: 1 Real MMK = 1 BO = 1 Game Point
- **Game Point Rules**:
  - If **Game Point** value = **100** → Supports up to **two decimal places**.
  - If **Game Point** value = **1** → **Does not support** decimal places.
- **Important:** If the **Operator** return an amount exceeding the supported decimal precision for the currency, the excess fraction may be **truncated** (lost). Please ensure that transfers comply with the supported decimal precision.
- For detailed decimal precision rules per currency, please contact the **Provider**'s support team.

Note on timestamp:

- All requests from the **Provider** will always include a **timestamp** field, which represents the moment the request was originally triggered by the **Provider** system.
- Depending on business rules and API context, the Operator **may choose to reject or accept requests** based on the timestamp value:
- ***For time-sensitive requests like***:
  - Bet
  - Withdraw
  - Join Tournament

⇨ If the timestamp is older than **10 minutes**, the Operator should reject the request to prevent stale or duplicate processing.

- ***For reliable delivery APIs (that follow retry mechanism), such as***:
  - Settle Bet
  - Cancel Bet
  - Bonus Win
  - Jackpot Win
  - Win Tournament
  - Cancel Tournament
  - Transaction
  - Deposit (for Fishing game)

⇨ These APIs may be re-sent **up to 24 hours** after the original event, and the timestamp remains **unchanged**. In such cases, the **Operator** must **accept** requests even if the timestamp is old, as part of the **idempotent** and **retry-safe** design.

5.1. Authenticate Token

This API is **triggered by the Provider**, and must be **implemented by the Operator**.
It is used to verify whether a player's session **token**—received when opening the game—is valid according to the **Operator's** system.

Flow Description:

1. The **player** clicks to open a game from the **Operator** platform.
2. The **Operator** initiates the request to the **Provider's** game entry point using the *Open Game URL* (**see section 4.1 Open Game URL**), passing in a **token** and other parameters.
3. Upon receiving the request, the **Provider** parses the **token** and immediately calls the **Operator's** /authenticate-token API to validate it.
4. The **Operator** checks the **token** and returns the result:
   - **Valid token** → **Provider** proceeds to open the game.
   - **Invalid token** → **Provider** returns an error and blocks access.

API Specification:

```
API URL: {operator-integration-api-url}/authenticate-token
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
appid=seamless&hash=1f4f020546a137427f4de3773dc04fc3&ip=128.199.64.190&times
tamp=1552970614889&token=xm6rhmjciweuq

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
     "Username": "DEMO",
     "Balance": 100.01,
     "Message": "Success",
     "Status": 0
}
```

Request Body:

Property	Type	Compulsory	Description
token	String	Y	**Unique identifier** of the player session. This token is obtained from the token parameter included in the Open Game URL *See section 4.1 Open Game URL*
ip	String	Y	IP address of the player
timestamp	Long	Y	Current UNIX timestamp in milliseconds.

			Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
appid	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String	Y	MD5 hash signature used to verify the integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Username	String	Y	The **unique identifier** of the player. Must be **4 to 32 alphanumeric characters**, **case-insensitive**.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.2. Authenticate Username/Password (Optional)

This API is triggered by the **Provider** when a player attempts to log in directly from the **Provider's** platforms (e.g., **website** or **mobile app**).

It enables the **Operator** to verify the player's login credentials (**username and password**) and decide whether to allow the user to access the **Provider's** system.

Flow Description:

1. The **player** enters their login credentials (**Username** and **Password**) on the **Provider**'s platform (e.g., website or mobile app).
   - To help the **Provider** determine which **AppId** the user belongs to, the **Username must follow this format**: AppId.Username (e.g., **TXA1.Demo**).
2. The **player** clicks the **Login** button.
3. The **Provider** sends the input data (**username**, **password**, **IP**, **timestamp**, etc.) and immediately calls the **Operator's** /authenticate API to validate it.
4. The **Operator** receives the request and verifies the credentials:
   - **Valid credentials** → **Provider** allows access.
   - **Invalid credentials** → **Provider** returns an error and blocks access.

API Specification:

```
API URL: {operator-integration-api-url}/authenticate
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
appid=seamless&hash=30795ed16135a0eb7711229c95670924&ip=128.199.64.190&passw
ord=abcdef&timestamp=1552976364273&username=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
     "Token": "xm6rhmjciweuq",
     "Balance": 100.05,
     "Message": "Success",
     "Status": 0
}
```

Request Body:

Property	Type	Compulsory	Description
username	String	Y	Player's username. Must be **4 to 32 alphanumeric characters**, case-insensitive.

			Does **not** include AppId.
password	String	Y	Player's login password
ip	String	Y	IP address of the player
timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
appid	String	Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String	Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*

Request Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Token	String	Y	**Unique identifier** of the player session. **Token has max length 64 characters**
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.3. Balance

This API **is triggered by the Provider** to retrieve the most up-to-date wallet balance of a player, either when the player opens a game or when the **Provider** needs to update the player's balance.

API Specification:

```
API URL: {operator-integration-api-url}/balance
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
appid=seamless&hash=45a8710d5db0aeabf4f3e3d881f732f8&timestamp=1554370334626
&username=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
timestamp	Long		Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
username	String		Y	Player's username. The value is always converted to **uppercase** by the Provider before being sent.

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.4. Gameplay APIs – Slot & Live Casino

APIs in this section are used during the gameplay of **slot games** and **live casino games**, including placing bets, settling results, handling bonus rewards, and managing special jackpot events.

Gameplay Flow Description:

1. **Start Round & Place Bet**
   - When a **player** starts a new round and places a wager (e.g., spin), the **Bet API** is triggered.
   - If the game supports multiple bet actions in the same round, each valid wager triggers the **Bet API** individually.

2. **Failed Bet Handling**
   - If any bet request fails due to timeout or undefined error, the **Cancel Bet API** must be triggered for that specific failed bet.

3. **Round Settlement**
   - If at least one valid bet exists in the round, the **Settle Bet API** must be called to finalize the round and credit winnings to the player.

4. **Bonus Rewards**
   - If bonus rewards such as **free spins**, **bonus features**, or **in-game jackpots** are triggered during the round, the **Bonus Win API** will be used to credit these winnings.

5. **Jackpot Events**
   - The **Jackpot Win API** is **not tied to a specific round**. It is triggered **independently** when the player wins a **random jackpot event** (e.g., *Angbao*, *Competition*), regardless of whether a game round has completed.

5.4.1. Bet

The **Bet API** is **triggered by the Provider** whenever a player places a wager in a game round (e.g., spinning a slot or placing a bet in live casino).

This **API deducts the specified bet amount** directly from the player's wallet balance at the time of the request.

API Specification:

```
API URL: {operator-integration-api-url}/bet
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
amount=0.50&appid=seamless&gamecode=zygj7oqga9nck&hash=41399ea6eeedd294fa0df
c0b911895e7&id=B_Tb5aa0018c11e&roundid=qc9cfdk7u5wca&timestamp=1552970894238
&username=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	**Unique identifier** for **a bet request**. This identifier must

				be unique **per player**. The value can be in **any string format** as long as it guarantees uniqueness for each bet made by the same player.
amount	Decimal	Positive numeric	Y	The amount the player wants to wager in this bet. *Must be a positive numeric value*.
username	String		Y	Player's username. The value is always converted to **uppercase** by the Provider before being sent
timestamp	Long		Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
gamecode	String		Y	The code representing the specific game in which the player is placing a bet. *please refer to section 4.2 List Game API*
roundid	String		Y	**Unique identifier** for the **game round**. Must be **unique per player**, and shared **across all bets** within the same round.

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.

Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.4.2. Settle Bet

The **Settle Bet API** is **triggered by the Provider** to finalize a game round and credit the outcome (**win** or **lose**) to the player.

This API must be called when **at least one valid bet exists** in the round, regardless of whether the player won or lost.

It ensures the **final result** of the round is recorded and balances are correctly updated on the **Operator** side.

Reliable Delivery: This API is part of the **critical financial flow** and follows the retry mechanism described in **Section 6.6 – Delivery & Retry Mechanism**.

Note: Once a game round is successfully settled, its result can be retrieved via the **Game History URL API** (**see Section 4.4 Game History URL**). This allows Operators to view round outcomes.

Special Note on Cancel After Settlement (Only Applies to Multi-Bet Rounds):

In **multi-bet rounds** (where a player can place multiple independent bets within the same round), a **Cancel Bet** request may be received **after** a **Settle Bet** has already been processed.

Why this can happen:

- **Network delays**.
- **Retry mechanism** (in case previous delivery attempts failed).

Even though the **Provider** always sends **Cancel Bet** for cancelled wagers, timing issues can cause a **Cancel Bet** to arrive late.

Operator Handling Guideline:

- In this case, the **Operator must refund** the player with the **exact amount of the canceled bet**.
- **Do not change** the round result — no updates are made to the previously processed settlement.
- Reason: The **Settle Bet** sent by the **Provider** only includes bets that were valid at the time of settlement.
- Therefore, the previously sent settlement amount is **final** and reflects the valid game state.

API Specification:

```
API URL: {operator-integration-api-url}/settle-bet
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
amount=1.00&appid=seamless&gamecode=zygj7oqga9nck&hash=ea28cd5801d5914e0
27d31275f5670e4&id=S_Tb5aa0018c11f&roundid=qc9cfdk7u5wca&timestamp=15529
70900488&username=DEMO&description=slot&type=main

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	**Unique identifier** for a settle request. This identifier must be unique **per player**. The value can be in **any string format** as long as it guarantees uniqueness for each settle made by the same player.
amount	Decimal	Positive numeric	Y	The total result amount of the game round. Always **≥ 0**. This represents the **final balance outcome** of the round. Used to determine **win/loss** as follows:

			• amount - total bet = 0 → **draw** • amount - total bet < 0 → **loss** • amount - total bet > 0 → **win**
username	String	Y	Player's username. The value is always converted to **uppercase** by the Provider before being sent
timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
gamecode	String	Y	Code of the game being settled. Must match the **gamecode** used in the Bet API.
roundid	String	Y	**Unique identifier** for the game round. Must match the value used in **Bet API**.
description	String	Y	Typically matches the **gamecode**. Used only for hash **verification purposes** and does not affect settlement processing logic.
type	String	Y	Typically matches the **gamecode**. Used only for hash **verification purposes** and does not affect settlement processing logic.

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.4.3. Cancel Bet

The Cancel Bet API is triggered by the Provider to cancel a previously submitted Bet API request that either:

- Failed **to process due to** timeout **or** unknown error.
- Did not receive a valid acknowledgment from the Operator.

This API ensures that funds are returned correctly to the player when a wager is deemed **invalid** or **unconfirmed**.

Reference for Rollback:

- **The** Cancel Bet API **includes the original** betID **to be cancelled**
- **The** Operator **must use this** betID **to:**
  - **Determine the** amount **that was previously wagered (if any).**
  - Perform a rollback to the player's balance accordingly.

Reliable Delivery: This API is part of the **critical financial flow** and follows the retry mechanism described in **Section 6.6 – Delivery & Retry Mechanism**.

Special Cases:

1. **Cancel After Settlement *(Only Applies to Multi-Bet Rounds)***

   In multi-bet rounds (where a player can place multiple independent bets within the same round), a Cancel Bet request may arrive after a Settle Bet has already been processed.

   Possible causes:
   - **Network delays**.
   - **Retry mechanisms** (from failed delivery attempts).

   Operator Handling Guideline:
   - **Refund** the player the **exact amount** of the cancelled bet.
   - Do **not change** the round result or alter the previously processed settlement.
   - Reason: The **Settle Bet** request sent by the **Provider** only includes bets valid at the time of settlement.
   - Therefore, the settlement amount previously sent is **final** and consistent with the valid game state.

2. **Cancel Bet Before Bet**

   Although the **Provider** only sends **Cancel Bet** after sending a **Bet** request, there are rare cases where the **Cancel Bet** may arrive **before** the actual **Bet** is received by the **Operator**.

   Possible causes:
   - **Network delays.**
   - **Packet loss.**
   - **Unconfirmed transmission** causing **uncertainty** on the **Provider's** side.

Operator Handling Guideline:

- **Accept and record** the **Cancel Bet** request even if the corresponding **Bet** has not yet been received.
- If the related **Bet** arrives later, **discard** it and **do not deduct** player balance, since the bet has already been invalidated by the earlier **Cancel Bet** request.

This ensures players are never charged for **uncertain** or failed bets, maintaining **consistency**.

API Specification:

```
API URL: {operator-integration-api-url}/cancel-bet
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
appid=seamless&gamecode=zygj7oqga9nck&hash=ea28cd5801d5914e027d31275f567
0e4&id=C_Tb5aa0018c11f&betid=B_Tb5aa0018c11e&roundid=qc9cfdk7u5wca&times
tamp=1552970900488&username=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	**Unique identifier** per user. The value can be in any string format

			as long as it guarantees uniqueness for each request made by the same player.
username	String	Y	Player's username, converted to **uppercase** by the Provider before sending.
timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
gamecode	String	Y	The game in which the bet occurred. Must match the **gamecode** from the Bet request.
roundid	String	Y	**Unique identifier** for the game round. Must be the same as in the corresponding Bet.
betid	String	Y	**Unique identifier** of the **Bet** being cancelled. Must match the **id** field from the corresponding **Bet request**.

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.4.4. Bonus Win

The **Bonus Win API** is used by the **Provider** to credit bonus winnings (e.g., free spins, bonus features) to a player's wallet.

Reliable Delivery: This API is part of the **critical financial flow** and follows the retry mechanism described in **Section 6.6 – Delivery & Retry Mechanism**.

API Specification:

```
API URL: {operator-integration-api-url}/bonus-win
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
amount=1.00&appid=seamless&gamecode=zygj7oqga9nck&hash=ea28cd5801d5914e027d3
1275f5670e4&id=Tb5aa0018c11f&roundid=qc9cfdk7u5wca&timestamp=1552970900488&u
sername=DEMO&description=bonus&type=bonus

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	**Unique identifier** per user. The value can be in any string format as long as it guarantees uniqueness for each request

				made by the same player.
amount	Decimal	Positive numeric	Y	**Bonus amount** to be credited (≥ 0.00).
username	String		Y	Player's username, converted to **uppercase** by the Provider before sending.
timestamp	Long		Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
gamecode	String		Y	Code of the game associated with the bonus.
roundid	String		Y	**Unique identifier** for the game round. Must be unique per player
description	String		Y	*See section 6.8 Type & Description for definition*.
type	String		Y	*See section 6.8 Type & Description for definition*.

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.4.5. Jackpot Win

The **Jackpot Win API** is used by the **Provider** to credit jackpot winnings to a player's wallet. This API is **not associated with a specific game round** and can be triggered independently (e.g., from events like **Angbao**, **Competitions**).

Reliable Delivery: This API is part of the **critical financial flow** and follows the retry mechanism described in **Section 6.6 – Delivery & Retry Mechanism**

API Specification:

```
API URL: {operator-integration-api-url}/jackpot-win
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
amount=1.00&appid=seamless&gamecode=zygj7oqga9nck&hash=ea28cd5801d5914e027d3
1275f5670e4&id=Tb5aa0018c11f&roundid=qc9cfdk7u5wca&timestamp=1552970900488&u
sername=DEMO&description=jackpot&type=jackpot

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	**Unique identifier** per user. The value can be in any string format as long as it guarantees

				uniqueness for each request made by the same player.
amount	Decimal	Positive numeric	Y	Jackpot amount to be credited (≥ 0.00).
username	String		Y	Player's username, converted to **uppercase** by the Provider before sending.
timestamp	Long		Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
gamecode	String		Y	Game code associated with the jackpot event. **Default (for generic jackpots): qc8y6dypyeboy**
roundid	String		Y	**Unique identifier** for the game round. Must be unique per player
description	String		Y	*See section 6.8 Type & Description for definition*.
type	String		Y	*See section 6.8 Type & Description for definition*.

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.5. Gameplay APIs – Fishing

Fishing games operate under a **wallet transfer model** that is significantly different from slot games. In fishing games, players are required to **transfer credit** into the **Provider's** system before initiating gameplay. This **in-game credit** is used to perform actions such as shooting fish, and the amount is directly deducted from the **Provider-side wallet**.

Credit Top-up

- The credit top-up is **not limited** — players can perform multiple **Withdraw API** calls during the session to increase their in-game balance as needed.
- Similarly, they can also **Withdraw** unused funds at any time by triggering the **Deposit API** to return money to the **Operator-side wallet**.

Automatic Deposit on Exit:

- When a player exits the game, if there is any remaining in-game credit, the **Provider** will **automatically perform a Deposit operation** to return the balance back to the **Operator**.

Gameplay Data Reporting

- During the session, the history of player actions (e.g., shooting records, total spent, total win) is **not reported in real time** per shot.
- Instead, the system **accumulates and groups gameplay data**, and sends it periodically **(typically once per minute)** to the **Operator** using the **Transaction API**.
- This ensures **efficient communication** while preserving **auditability** and **traceability**.

5.5.1 Transaction

The **Transaction API** is used by the **Provider** to **send aggregated gameplay data** during a **Fishing Game session**. Unlike slot games where each bet is sent individually, fishing games involve **continuous actions** (e.g., shooting) that are **grouped** and **reported periodically**.

Reporting Interval

This API is called at a **regular Interval** (e.g., every **60 seconds**) to update the **Operator** with:

- **The amount spent** (total bet value)
- **The result** (total win)
- **Starting and ending balance**
- **Round identifiers**

This API does **not affect player balance**. It is used **only for logging and auditing**.

Reliable Delivery: This API is part of the **critical financial flow** and follows the retry mechanism described in **Section 6.6 – Delivery & Retry Mechanism**

API Specification:

```
API URL: {operator-integration-api-url}/transaction
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
amount=6.00&appid=seamless&description=Main&endbalance=318.00&gamecode=kk8nq
m3cfwtng&hash=00b319f583ce6b2a26c2df0fe75f0d47&id=xm6qbj99cmfdq&result=4.00&
roundid=xm6qbj99cmfdq&startbalance=320.00&timestamp=1555474256669&type=Main&
username=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	Unique transaction ID. Must be unique per player.
amount	Decimal	Positive numeric	Y	Total amount spent (≥ 0.00).
result	Decimal	Positive numeric	Y	Total win result of the session (≥ 0.00). result - amount = 0 → **draw** result - amount < 0 → **loss** result - amount > 0 → **win**
username	String		Y	Player's username, converted to

				uppercase by the Provider before sending.
timestamp	Long		Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
gamecode	String		Y	The code representing the specific fishing game in which the session took place. *please refer to section 4.2 List Game API*
roundid	String		Y	**Unique identifier** for the game round. Must be unique per player
description	String		Y	Value: **Main** – Indicates **a primary gameplay transaction**. Used only for **hash verification** purposes and does not affect **processing logic**.
type	String		Y	Value: **Main** – Indicates **a primary gameplay transaction**. Used only for **hash verification** purposes and does not affect **processing logic**.
startbalance	Decimal	Positive numeric	Y	Player balance at the **start** of the game round.
endbalance	Decimal	Positive numeric	Y	Player balance at the **end** of the game round.

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success",

			"Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.5.2 Withdraw

The **Withdraw API** is used by the **Provider** to **transfer credits from the Operator to the Provider** at the beginning — or anytime during — a **Fishing Game session**.

Unlike traditional games where credits are deducted per action (e.g., spin), fishing games require **credit top-ups** into the **Provider's wallet** before the player can engage in shooting.

This API can be called **multiple times during gameplay**, allowing players to add more credits as needed without restarting the session.

This API directly deducts the specified amount from the Operator's system and makes it available inside the Provider's system.

Handling Timeout or Unknown Errors (Withdraw Reliability)
If the Provider's **Withdraw API** call to the Operator **timeout** or returns an **unknown error**, the Provider will:

1. **Retry the request immediately**, up to **3 times**.
2. If **all retries fail**, the transaction is considered **unsuccessful**, and **no credit is recorded** on the **Provider** side.

However, the **Operator** may have **already deducted the funds** despite the **Provider** not recording the transaction.
In such cases, the **Operator** must use **API 4.7 – Verify Withdraw Transaction** to confirm the transaction status.

- If the transaction **does not exist** on the **Provider** system, the **Operator** is responsible for **rolling back** the deducted amount to the player.

This mechanism ensures **consistency between both systems** and prevents users from losing money.

API Specification:

```
API URL: {operator-integration-api-url}/withdraw
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
amount=1.00&appid=seamless&hash=ea28cd5801d5914e027d31275f5670e4&id=Tb5aa001
```

```
8c11f&timestamp=1552970900488&username=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	**Unique identifier** for a withdraw request. This identifier must be unique **per player**. The value can be in **any string format** as long as it guarantees uniqueness for each withdraw made by the same player.
amount	Decimal	Positive numeric	Y	The amount to be withdrawn from the Operator and credited to the Provider's in-game wallet. Must be a positive number (e.g., 1.00, 100.50).
username	String		Y	The player's username. This value is always **converted to uppercase** by the Provider before sending.
timestamp	Long		Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.5.3 Deposit

The **Deposit API** is used by the **Provider** to return **unused in-game credits** from the fishing game session back to the **Operator's** wallet.

This API is typically triggered when the player **exits the game**, but can also be **manually triggered by the player** at any time (e.g., by using the **withdraw feature** provided in the game client). This gives players flexibility to **reclaim their balance** without ending the session.

Reliable Delivery: This API is part of the **critical financial flow** and follows the retry mechanism described in **Section 6.6 – Delivery & Retry Mechanism**.

API Specification:

```
API URL: {operator-integration-api-url}/deposit
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
amount=1.00&appid=seamless&hash=ea28cd5801d5914e027d31275f5670e4&id=Tb5aa001
8c11f&timestamp=1552970900488&username=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	**Unique identifier** for a deposit request. This identifier must be unique **per player**. The value can be in **any string format** as long as it guarantees uniqueness for each deposit made by the same player.
amount	Decimal	Positive numeric	Y	The balance to be returned from the **Provider** to the **Operator**. Must be ≥ **0.00**.
username	String		Y	The player's username. This value is always **converted to uppercase** by the Provider before sending.
timestamp	Long		Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.

Request Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.

Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.6. Tournament APIs

The **Tournament APIs** enable players to join and participate in various tournaments hosted by the **Provider's gaming system**.

5.6.1 Join Tournament

The **Join Tournament API** is triggered when a player enters a tournament **event.** The **Operator** is required to deduct the tournament entry fee from the **player's balance and** record the **transaction.**

Depending on the **tournament configuration**, a player may be allowed to join the same tournament multiple times.

Each **participation** is considered a separate entry, and the Provider will generate a unique **join record ID** for each entry.

API Specification:

```
API URL: {operator-integration-api-url}/join-tournament
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
amount=10.00&appid=seamless&extendedinfo=%7B%22PoolContribution%22%3A2.00%7D
&hash=c58166e6524e28cbdb47037133a8f83a&id=jxeahin3up4q51&timestamp=162564481
4173&tournamentid=xeahin3up4q51&username=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	**Unique identifier** per user. The value can be in any string format as long as it guarantees uniqueness for each request made by the same player.
tournamentid	String		Y	The **unique identifier** of the tournament that the player is joining. *Please refer to Section 4.13.2 – Get List Tournaments* to retrieve available tournament IDs.
amount	Decimal	Positive numeric	Y	Entry fee to be deducted from the player's balance. Must be a positive number
username	String		Y	The player's username. This value is always **converted to uppercase** by the Provider before sending.
timestamp	Long		Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
`extendedinfo`	String (JSON, URL-encoded)		N	Optional **metadata** for the join (e.g., {"PoolContribution":2.00}).

			Send as **URL-encoded** string. **- Do not use this property in Hash** - Is JSON string. Client can deserialize this value to object if need to consume. For example: JSON.parse(value)

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.6.2 Cancel Join Tournament

The **Cancel Join Tournament API** is **triggered** by the **Provider** to **cancel** a previously submitted **Join Tournament** request that:

- **Failed** due to timeout or unknown error.
- Did not receive a valid **acknowledgment** from the Operator.

This API ensures that the **entry fee** is **refunded** correctly to the **player** when a tournament registration is deemed **invalid** or **unconfirmed**.

Reference for Rollback:

The **Cancel Tournament API includes** the original ***jointournamentid*** (i.e., the **id** from the **Join Tournament** request) to be canceled.

The **Operator** must use this **ID** to determine whether **a deduction** was made and, if so, perform **a rollback** to the **player's balance** accordingly.

Reliable Delivery: This API is part of the **critical financial flow** and follows the retry mechanism described in **Section 6.6 – Delivery & Retry Mechanism**.

Handling Edge Cases: Cancel Before Join

Although the **Provider** only triggers **Cancel Join Tournament** after a **Join Tournament** request, there may be rare situations where the **Cancel** arrives **before** the actual **Join** reaches the **Operator**.

Possible causes:

- **Network latency**
- **Packet loss**
- **Transmission uncertainty** on the **Provider's** side

Operator Handling Guideline:

- Accept and record the **Cancel Join Tournament** request even if the corresponding **Join Tournament** has not yet been received.
- If the related **Join Tournament** request arrives later, **ignore it** and **do not deduct** the player's balance, as the transaction has already been invalidated.

This mechanism ensures that players are never charged for **uncertain** or **failed** tournament registrations, maintaining **consistency**.

API Specification:

```
API URL: {operator-integration-api-url}/cancel-join-tournament
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
appid=seamless&extendedinfo=%7B%7D&hash=9bbf0aacc55559c922343588229dd905&id=
cxeahin3up4q51&jointournamentid=jxeahin3up4q51&timestamp=1625645436243&usern
ame=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.

hash	String	Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String	Y	**Unique identifier** per user. The value can be in any string format as long as it guarantees uniqueness for each request made by the same player.
jointournamentid	String	Y	The **id** value from the original **Join Tournament** request that needs to be canceled.
username	String	Y	The player's username. This value is always **converted to uppercase** by the Provider before sending.
timestamp	Long	Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
extendedinfo	String (JSON, URL-encoded)	N	Optional **metadata** for the cancel join. - **Do not use this property in Hash** - Is JSON string. Client can deserialize this value to object if need to consume. For example: JSON.parse(value)

Response Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).

Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

5.6.3 Win Tournament

The **Win Tournament** API is **triggered by the Provider** when a tournament ends and a player is declared a **winner**.

Summary – Win Tournament Handling

- Players may **join tournaments multiple times**, and thus can **win multiple times**.
- The **Provider** will send **separate Win Tournament requests** for each winning entry, each with a **unique Win ID**.
- In tournaments using **ExclusivePass** (managed entirely by the **Provider**), the **Operator** will **not** receive any **Join Tournament** requests.
- However, if a player wins, the **Provider** will still send a valid **Win Tournament** request.

Operator must:

- **Accept all Win Tournament requests**, even if no corresponding **Join Tournament** was received.
- **Credit** the prize amount as instructed.
- **Note:** All prizes are **fully funded by the Provider**.

API Specification:

```
API URL: {operator-integration-api-url}/win-tournament
HTTP Method: POST
Content-Type: application/x-www-form-urlencoded

Request Content:
amount=5000.00&appid=seamless&extendedinfo=%7B%22PoolContribution%22%3A2.00%
7D&hash=c9d19dcb9cea559c3d111c1779002534&id=wxeahin3up4q51&timestamp=1625646
194210&tournamentid=xeahin3up4q51&username=DEMO

Response body: HTTP/1.1 200 OK
Content-Type: application/json
{
    "Balance":1186.11,
    "Message":"Success",
    "Status":0
}
```

Request Body:

Property	Type	Value	Compulsory	Description
appid	String		Y	Created by the **Provider**. Each

				currency corresponds to a unique **AppID**. Used to identify which game environment the player belongs to.
hash	String		Y	MD5 hash signature used to verify the Integrity of the request. The signature is generated based on sorted key-value pairs and a secret key. *See section 6.2 Hash for definition*
id	String		Y	**Unique identifier** per user. The value can be in any string format as long as it guarantees uniqueness for each request made by the same player.
tournamentid	String		Y	The **unique identifier** of the tournament that the player won..
username	String		Y	The player's username. This value is always **converted to uppercase** by the Provider before sending.
amount	Decimal	Positive numeric	Y	The prize amount to be credited to the player. Must be a positive number (e.g., 100.00).
timestamp	Long		Y	Current UNIX timestamp in milliseconds. Used for request validation and replay protection. *See section 6.1 Timestamp for definition*.
extendedinfo	String (JSON, URL-encoded)		N	Optional **metadata** for the join (e.g., {"PoolContribution":2.00}). Send as **URL-encoded** string. - **Do not use this property in Hash** - Is JSON string. Client can deserialize this value to object if need to consume. For example: JSON.parse(value)

Request Body:

Property	Type	Compulsory	Description
Status	Integer	Y	Indicates the execution result of the API. A value of 0 means success; any non-zero value indicates an error for *See Section 6.3 – Status Code* for more details.
Message	String	Y	Descriptive message explaining the response result (e.g., "Success", "Invalid Token", etc.).
Balance	Decimal	Y	The player's current balance, returned with **up to 2 decimal places**.

6 Appendix

6.1 Timestamp

The **timestamp** is used as part of request to verify **Integrity**. If the request's timestamp is significantly older than the current time, the **Operator** should reject the request.

The **timestamp** is defined as the number of **milliseconds** elapsed since midnight **Coordinated Universal Time (UTC)** of **January 1, 1970**.

Sample Code – C# (Get Current Timestamp)

```
public static readonly DateTime UnixEpoch = new DateTime(1970, 1, 1, 0, 0, 0, 0,
DateTimeKind.Local);
public static long GetCurrentTimestamp()
{
	return (long)DateTime.UtcNow.Subtract(UnixEpoch).TotalMilliseconds;
}
```

Notes:

1. All **timestamps** in the API are expected to be in **UNIX timestamp format** (milliseconds).
2. The **Operator** should ensure the difference between the provided timestamp and the current time is within the allowed range to prevent **replay attacks**.

6.2 Hash

The **hash** is an **MD5-based signature** used to ensure the **Integrity** and **authenticity** of requests sent between the **Provider** and **Operator** systems.

This mechanism helps prevent **tampering** or **replay attacks** by verifying that the request data was not altered in transit and was generated by an authorized party.

Every request must include a **hash** parameter, which is calculated by the sender using a **shared secret key** and the contents of the request. The receiver will independently compute the hash using the same logic and compare it to the received value to validate the request.

Signature Generation Steps:

1. **Remove empty parameters**: Omit any key-value pairs where the value is empty or null.
2. **Convert all keys to lowercase:** Keys must be converted to lowercase before sorting and concatenation.
3. **Sort keys in alphabetical order:** After converting to lowercase, sort all keys **alphabetically** (a–z).
4. **Format numeric values:** Format decimal numbers to **two decimal places**, e.g., **1.00**.
5. **Construct query string:** Concatenate the key-value pairs into a query string format: key1=value1&key2=value2&...
6. **Append the secret key:** Add the shared secret key directly at the end of the string: *key1=value1&key2=value2MySecretKey123*
7. **Apply MD5 hash:** Generate the **MD5** hash of the resulting string using UTF-8 encoding.
8. **Convert to lowercase hexadecimal:** Output should be a 32-character lowercase hexadecimal string.

6.2.1 Hash Generation Logic (C#):

```
 using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using System.Text;

/// <summary>
/// C# program to generate MD5 hash for Seamless Wallet API signature
/// </summary>
class Program
{
    static void Main(string[] args)
    {
        ////////////////////////////////////////////////////
        // ▶ Example usage
        ////////////////////////////////////////////////////

        var parameters = new Dictionary<string, string>
        {
            { "AppID", "TXA2M" },
            { "Timestamp", "1561025342810" },
            { "Amount", "1000.0" },
            { "ID", "xe6jrzgntw1pc" },
            { "Username", "DEMO" }
        };
```

```
        string secretKey = "123456";

        // ▶ Run the hash function
        string hash = GenerateMD5Hash(parameters, secretKey);

        // ▶ Print the result
        Console.WriteLine("Generated Hash: " + hash);

        ////////////////////////////////////////////////////
        // ▶ Expected Output:
        // Raw string before hashing:
        //
amount=1000.00&appid=TXA2M&id=xe6jrzgntw1pc&timestamp=1561025342810&username=DEM
O123456
        //
        // MD5 Hash (UTF-8):
        // f38c4df2641a9c40129ae039a9ef8def
        ////////////////////////////////////////////////////
    }

    /// <summary>
    /// Generate MD5 hash from sorted parameters and secret key
    /// </summary>
    public static string GenerateMD5Hash(Dictionary<string, string> parameters,
string secretKey)
    {
        // 1. Remove empty values and convert keys to lowercase
        var filtered = parameters
            .Where(kv => !string.IsNullOrWhiteSpace(kv.Value))
            .ToDictionary(kv => kv.Key.ToLower(), kv => kv.Value);

        // 2. Format amount (if exists)
        if (filtered.ContainsKey("amount") &&
decimal.TryParse(filtered["amount"], out decimal amountValue))
        {
            filtered["amount"] = amountValue.ToString("0.00");
        }

        // 3. Sort keys alphabetically
        var sorted = filtered.OrderBy(kv => kv.Key);

        // 4. Build raw string
        var rawString = string.Join("&", sorted.Select(kv =>
$"{kv.Key}={kv.Value}"));

        // 5. Append secret key
        rawString += secretKey;

        // 6. Hash using MD5
        using (var md5 = MD5.Create())
        {
            byte[] inputBytes = Encoding.UTF8.GetBytes(rawString);
            byte[] hashBytes = md5.ComputeHash(inputBytes);
            return string.Concat(hashBytes.Select(b => b.ToString("x2")));
        }
    }
}
```

6.2.2 Hash Generation Logic (PHP):

```php
<?php

/**
 * Generate MD5 Hash for Seamless Wallet signature
 *
 * @param array $params Key-value parameters to be hashed
 * @param string $secretKey Shared secret key to append
 * @return string MD5 hash (lowercase)
 */
function generateMD5Hash(array $params, string $secretKey): string {
    // 1. Remove empty values and convert keys to lowercase
    $filtered = [];
    foreach ($params as $key => $value) {
        if (trim($value) !== '') {
            $filtered[strtolower($key)] = $value;
        }
    }

    // 2. Format 'amount' to two decimal places if applicable
    if (isset($filtered['amount']) && is_numeric($filtered['amount'])) {
        $filtered['amount'] = number_format((float)$filtered['amount'], 2, '.',
'');
    }

    // 3. Sort keys alphabetically
    ksort($filtered);

    // 4. Build the raw string: key=value&key=value...
    $rawString = '';
    foreach ($filtered as $key => $value) {
        if ($rawString !== '') {
            $rawString .= '&';
        }
        $rawString .= $key . '=' . $value;
    }

    // 5. Append the secret key
    $rawString .= $secretKey;

    // 6. Generate MD5 hash
    return strtolower(md5($rawString));
}

//////////////////////////////////////////////////
// ▶ Example usage
//////////////////////////////////////////////////

$params = [
    'AppID' => 'TXA2M',
    'Timestamp' => '1561025342810',
    'Amount' => '1000.0',
    'ID' => 'xe6jrzgntw1pc',
    'Username' => 'DEMO'
];
```

```
$secretKey = '123456';

// ▶ Call the function
$hash = generateMD5Hash($params, $secretKey);

// ▶ Print result
echo "Generated Hash: " . $hash . PHP_EOL;

// ▶ Expected Output:
// The raw string becomes:
//
amount=1000.00&appid=TXA2M&id=xe6jrzgntw1pc&timestamp=1561025342810&username=DEM
O123456
// MD5 hash of above = f38c4df2641a9c40129ae039a9ef8def
```

6.2.3 Hash Generation Logic (Java):

```
import java.security.MessageDigest;
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.util.Locale;

/**
 * This program demonstrates how to generate an MD5 hash signature
 * for Seamless Wallet API requests.
 */
public class SeamlessHashExample
{

    /**
     * Generate MD5 hash from parameters + secret key
     */
    public static String generateMD5Hash(Map<String, String> params, String
secretKey)
    {
        Map<String, String> filtered = new HashMap<>();

        // 1. Remove empty values and convert keys to lowercase
        for (Map.Entry<String, String> entry : params.entrySet())
        {
            String key = entry.getKey().toLowerCase(Locale.ROOT);
            String value = entry.getValue();

            if (value != null && !value.trim().isEmpty())
            {
                // 2. Format 'amount' to 2 decimal places
                if (key.equals("amount"))
                {
                    try
                    {
                        double d = Double.parseDouble(value);
                        value = String.format(Locale.ROOT, "%.2f", d);
```

```
                    }
                    catch (NumberFormatException ignored)
                    {
                        // Ignore invalid number format
                    }
                }
                filtered.put(key, value);
            }
        }

        // 3. Sort keys alphabetically
        List<String> sortedKeys = new ArrayList<>(filtered.keySet());
        Collections.sort(sortedKeys);

        // 4. Build raw string
        StringBuilder raw = new StringBuilder();
        for (String key : sortedKeys)
        {
            if (raw.length() > 0)
            {
                raw.append("&");
            }
            raw.append(key).append("=").append(filtered.get(key));
        }

        // 5. Append secret key
        raw.append(secretKey);

        // 6. Generate MD5 hash
        try
        {
            MessageDigest md = MessageDigest.getInstance("MD5");
            byte[] hashBytes =
md.digest(raw.toString().getBytes(StandardCharsets.UTF_8));

            StringBuilder hex = new StringBuilder(hashBytes.length * 2);
            for (byte b : hashBytes)
            {
                hex.append(String.format("%02x", b));
            }

            return hex.toString();
        }
        catch (Exception e)
        {
            throw new RuntimeException("MD5 hashing failed", e);
        }
    }

    public static void main(String[] args)
    {
        ////////////////////////////////////////////////////
        // ▶ Example usage
        ////////////////////////////////////////////////////
        Map < String, String > params = new HashMap<>();
        params.put("AppID", "TXA2M");
        params.put("Timestamp", "1561025342810");
```

```
        params.put("Amount", "1000.0");
        params.put("ID", "xe6jrzgntw1pc");
        params.put("Username", "DEMO");

        String secretKey = "123456";

        // ▶ Run hash function
        String hash = generateMD5Hash (params, secretKey);

        // ▶ Print the result
        System.out.println("Generated Hash: " + hash);

        ////////////////////////////////////////////////////
        // ▶ Expected Output:
        // Raw string before hashing:
        //
amount=1000.00&appid=TXA2M&id=xe6jrzgntw1pc&timestamp=1561025342810&username=DEM
O123456
        //
        // Hash (MD5, UTF-8):
        // f38c4df2641a9c40129ae039a9ef8def
        ////////////////////////////////////////////////////
    }
}
```

6.3 Status Code

Status	Description	Applicable APIs
0	Success	All
1	IP not allowed	All
2	Invalid AppID	All
3	Invalid token	Authenticate-Token
4	Invalid parameters (Example: **gameCode** is invalid or not supported)	All
5	Invalid signature	All
6	Invalid timestamp	All
7	Invalid Username or Password	Authenticate
100	Insufficient fund	Bet, Withdraw, Join Tournament
201	Transaction is being processed	All
999	Server under maintenance	All

1000	Other (Uncategorized error)	All

6.4 Datetime Format (Global Convention)

All **datetime** values returned by the **Provider** follow the **ISO 8601 format**. Depending on the API and field, the format may include **seconds**, **milliseconds**, **microseconds**, and/or **timezone** information.

Supported datetime patterns:

- yyyy-MM-dd'T'HH:mm:ss Example: 2025-08-05T15:30:00
- yyyy-MM-dd'T'HH:mm:ss.fff Example: 2025-08-05T15:30:00.123
- yyyy-MM-dd'T'HH:mm:ss.ffffff Example: 2025-08-05T15:30:00.789424
- yyyy-MM-dd'T'HH:mm:ss.fff+08:00 Example: 2025-08-05T15:30:00.456+08:00

Important Notes:

- All **datetime** values are based on **GMT+8 timezone** unless otherwise stated.
- The **datetime** precision (to **second**, **millisecond**, or **microsecond**) may vary depending on the API.
- **Clients** should implement **flexible ISO 8601 parsers** that can handle optional fractional seconds and **timezone** information.

6.5 Language Format (Global Convention)

All APIs that require a language input or return localized content use **ISO 639-1** two-letter language codes.

Supported language:

Code	Language
en	English
zh	中文 (Chinese)
th	ไทย (Thai)
id	Bahasa Indonesia
ms	Bahasa Melayu

Notes: If the provided language code is not supported, the default response will be in **English** (en).

6.6 Delivery & Retry Mechanism

To ensure **accuracy** and **consistency** in critical game and wallet operations, a **reliable delivery mechanism** is applied to all **sensitive APIs**.

If any of the APIs listed above do **not** receive a successful response **(Status = 0)** from the **Operator**, the **Provider** will retry the request using the following schedule:

General Retry Rule (Default for Most APIs**)**

Applies to:

- 5.4.2 Settle Bet
- 5.4.3 Cancel Bet
- 5.4.4 Bonus Win
- 5.4.5 Jackpot Win
- 5.5.1 Transaction
- 5.5.3 Deposit
- 5.6.2 Cancel Join Tournament
- 5.6.3 Win Tournament

Retry Logic:

- **Retry up to 5 times**, every **10 minutes**
- **Final retry** after **24 hours**
- If **all retries fail**, request is **logged** in the **Back Office (BO)** for **manual re-trigger** by **Operator**

6.7 Game Type

The following table lists the supported game types and their descriptions:

GameType	Description
Slot	A spin-based game where players match symbols on reels to win. Includes **bonus** rounds, **free spins**, and **jackpot** features.
Fishing	An interactive shooting-style game where players aim and fire to catch fish and earn points or payouts. Often features **boss battles** and **special weapons**.
Bingo	A number-matching game where players mark called numbers on their cards. Wins are based on completing specific patterns first.
Crash	A high-risk, high-reward game where a multiplier increases over time, and players must cash out before the multiplier "**crashes**" to secure their winnings.
ECasino	A broad category of electronic games that includes **mini-games**, **arcade styles**, or digital versions of traditional gambling games not covered by other categories.

6.8 Type & Description Mapping

a. Bonus API

Description	Type	Notes
StartFreeSpin	Main	Trigger event indicating the start of a free spin round.
FreeSpin	Main	Indicates a spin performed during a free spin round.
LastFreeSpin	Main	Indicates the final spin in a free spin round.
SelectFreeSpin	Bonus	Special bonus selection event within a free spin feature.
Bonus	Bonus	Bonus feature trigger (non-free spin).
Fever	Fever	Fever mode feature trigger.
Main	Main	Special FreeSpin feature trigger.
Special	Promotion	Special FreeSpin from Promotion
Jackpot	PJPGRAND	Progressive Jackpot – Grand
Jackpot	PJPMAJOR	Progressive Jackpot – Major
Jackpot	PJPMINOR	Progressive Jackpot – Minor
Jackpot	PJPMINI	Progressive Jackpot – Mini
BonusCode	Main	If this is a freeSpin request with the BonusCode added from the "Member Add FreeSpin" API

b. Jackpot API

Description	Type	Notes
<Event Name> - Angbao	<Event Name> - Angbao	Angbao Series – Special jackpot event. The **<Event Name>** suffix varies by event (e.g., Halloween, 7/7, Lunar New Year, Double 0808, Christmas, etc.).
Powerbar	Powerbar	Powerbar event
Prize Drop	Prize Drop	Prize Drop event
Competition	Competition	Competition event

7 FAQ

1. **How "Bet" is associated with "Settle"?**

   A Settle can be for multiple Bet. Bet and Settle are associated by RoundID.

2. **When to use the "Bonus Win" API?**

   The Bonus can be thought as Settle without Bet. Third party should add balance to player when receive the event.

3. **When to use the "Jackpot Win" API?**

   The Jackpot can be thought as Settle without Bet. Third party should add balance to player when receive the event.

4. **Can “Cancel” be received after “Settle”?**

   Yes. There is case that Cancel will be sent after Settle. In this case, the balance should be added back to player. Third party should adjust transaction.

5. **When to use the “Withdraw” API?**

   This API is used for to player transfer money to play game Fish.

6. **When to use the “Deposit” API?**

   This API is used for to return money back.

7. **When to use the “Transaction” API?**

   This API is used for to send bet detail grouping transaction (Fish) in which the bet detail is not sent when Bet. The API has no impact on the player balance.

8. **When to use the “Transaction is being processed” status code?**

   When Provider sends the first request to Third party but it processed longer than the timeout, Provider will re-send again.

   a. The first request is being processed -> the second request should be returned with the status **201**
   b. The first request is completed -> the second request should be returned with the correct status (which makes the call idempotent).